



Плюсы и минусы микросервисной архитектуры

Microservice Architecture



Проверить, идет ли запись

Меня хорошо видно & слышно?



Плюсы и минусы микросервисной архитектуры



Наталья Чалкова

- Руководитель направления ИТ-архитектуры ФКУ «ГосТех»
- Более 15 лет опыта Java-разработчиком и TeamLead
- Более 4 лет опыта по проектированию архитектуры
- Заказная разработка Enterprise систем, проектирование и разработка ГИС



natalia.chalkova@gmail.com
natalia_chalkova



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе
#OTUS Microservice-2026-08



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом

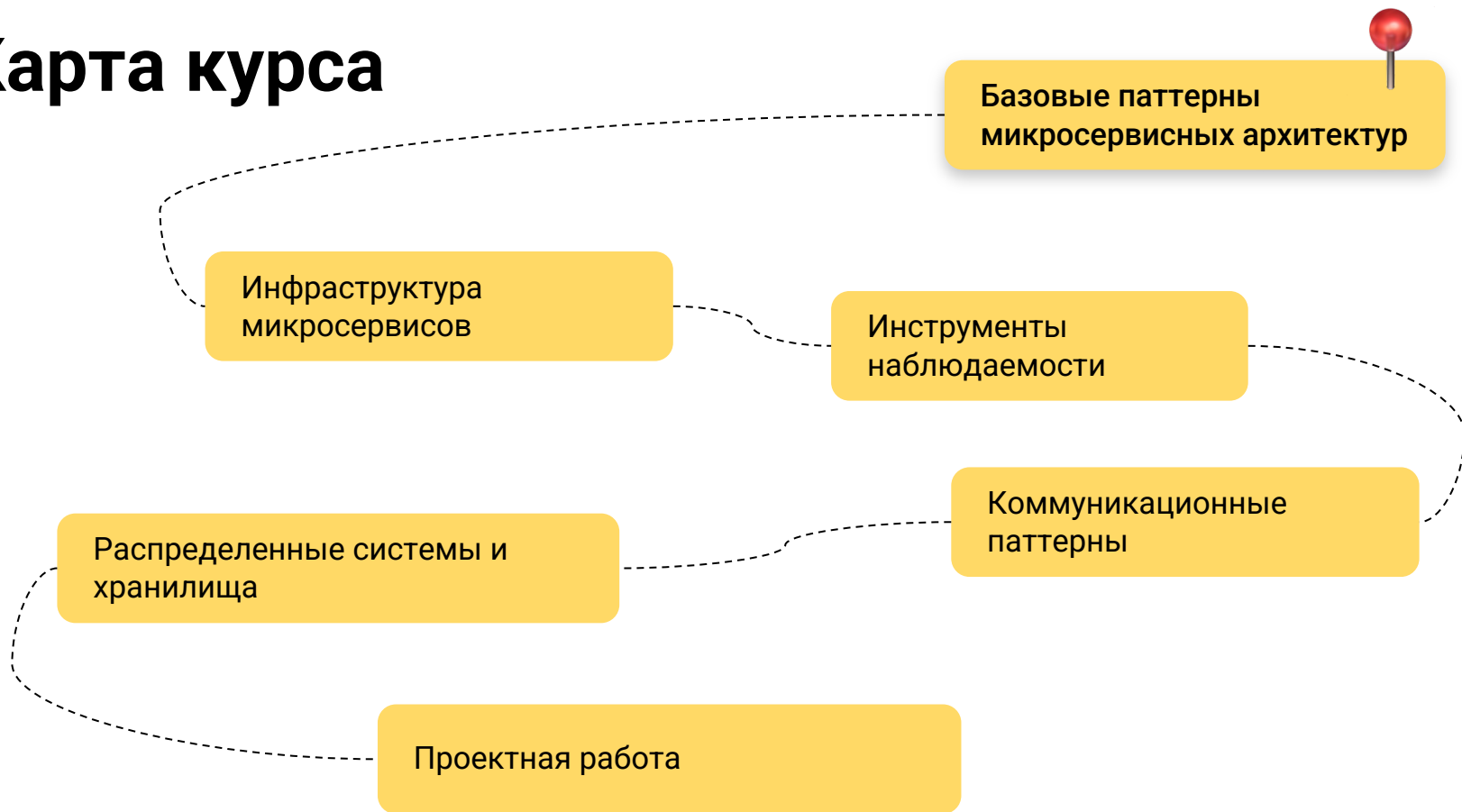


Документ



Ответьте себе или
задайте вопрос

Карта курса



Маршрут вебинара

Архитектура и роль архитектора ПО

Проблемы монолита

Проблемы микросервисов

Обзор основных паттернов
в микросервисной архитектуре
(обзор курса)

Проектная работа и домашние
задания

Цели вебинара

1. Узнать плюсы и минусы монолитной архитектуры.
2. Узнать плюсы и минусы микросервисной архитектуры.
3. Ознакомление с разделами курса, проблематикой и паттернами микросервисной архитектуры.

Смысл

Зачем вам это уметь

1. Выбирать подходящий вариант архитектуры информационной системы.
2. Учитывать достоинства и недостатки микросервисной архитектуры при проектировании информационной системы.

Архитектура



Что такое **архитектура** в контексте разработки программного обеспечения?

Архитектура

- **Архитектура** – это набор (и процесс принятия) архитектурных решений.
- **Архитектурные решения** – это решения, которые потом сложнее всего будет изменить, стоимость ошибки которых максимальна (фундаментальные решения).

Архитектура

Архитектура – это набор, связанных между собой структур, которые определяют свойства системы.

Структуры:

- Модули – программные блоки.
- Компоненты-и-коннекторы – компоненты и способы их взаимодействия
- Структуры размещения - размещение элементов системы во внешних средах

И соответствующие им различные представления.

Типы архитектур

Архитектура ПО описывает элементы и взаимодействия информационной системы, включая слой программных элементов(модулей) и инфраструктуры(железо и сети).

Типы архитектур:

- монолитная архитектура
- сервис-ориентированная (SOA)
- микросервисная архитектура



Монолит



SOA



Микросервисы

Архитектор ПО

Архитектор ПО

Архитектор ПО обеспечивает жизненный цикл архитектурных решений.

- Проектирование и оценка архитектурного решения
- Оформление и презентация
- Сопровождение и консультации

Проектирование архитектуры

- **Архитектурный анализ**

Общение со стейкхолдерами и бизнесом, получение архитектурно значимых требований, выработка совместного продуктового и технического решения

- **Архитектурный синтез**

Придумать архитектурное решение

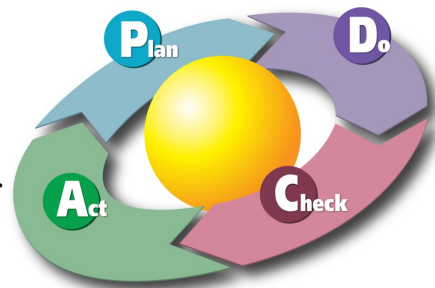
- **Оценка архитектуры**

Оценить характеристики получившегося решения

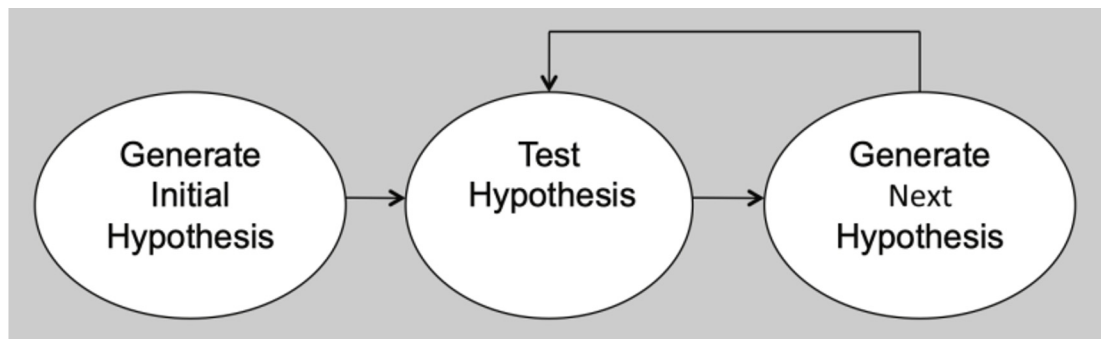
Проектирование архитектуры

Проектирование архитектуры представляет собой итеративный процесс.

Похожий на PDCA цикл Деминга.

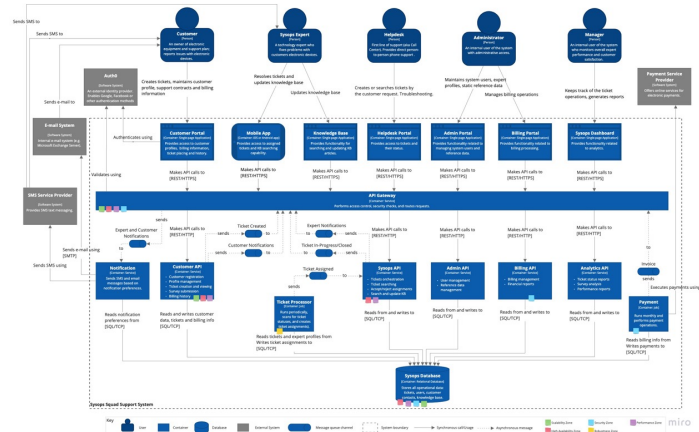
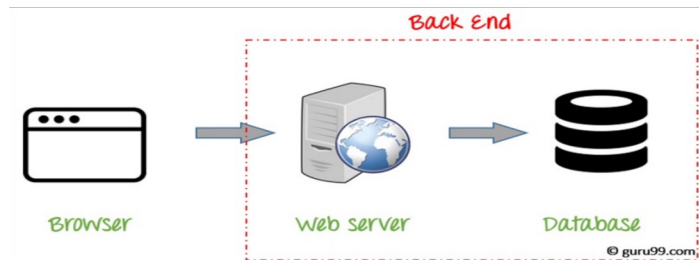


- Генерируем первоначальную архитектуру (и гипотезу) на основе AS
- Тестируем архитектуру: на основе представлений архитектуры делаем оценку атрибутов качества
- Получаем обратную связь, генерируем следующую гипотезу



Оформление и презентация архитектурного решения

- **Презентация решения бизнесу**
Какие есть развилки по решениям
- **Презентация решения разработке**
Донести и «продать» основное техническое видение до исполнителей (людей или команд).
(C4 model, UML диаграммы и др.)



Сопровождение архитектурного решения

- **Техническое лидерство** и консультации
- **Внесение изменений** при необходимости
- **Координация исполнителей** и команд разработки при необходимости
- **Архитектурный надзор**

Вопросы?

 если есть вопросы

 если вопросов нет

Монолитная архитектура



С какой архитектурой сейчас работаете?

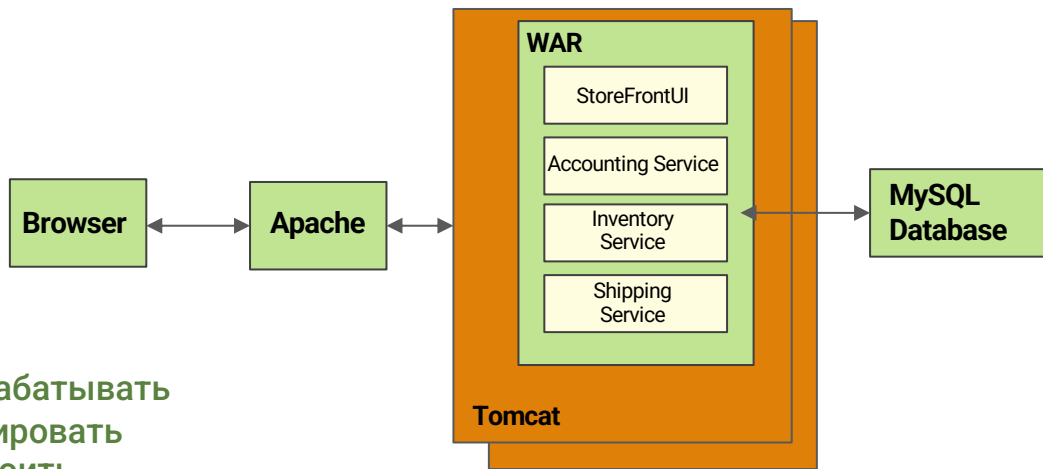
1 – Микросервисная

2 – Монолит, сервисная (SOA)

Монолит

Монолит – паттерн разработки, когда приложение проектируется, разрабатывается и деплоится как единое целое.

Традиционная архитектура веб-приложений



Легко:

- разрабатывать
- тестировать
- деплоить
- масштабировать

Проблемы монолита

Сценарий масштабируемости разработки



Что будет происходить с монолитом при увеличении бизнес логики, количества разработчиков и размера кодовой базы?

Проблемы монолита

Примеры проблемных сценариев:

- **Обновление фреймворка, используемых библиотек, языка программирования с ростом размера кодовой базы становится сложнее.**

Чем больше монолит, тем выше риски изменений и тем больше изменений необходимо произвести. С какого-то момента обновление практически невозможно.

- **Ошибка в одном из модулей может привести к падению всего приложения.**

Например, если модуль стал течь по памяти или CPU. Чем больше монолит, тем выше риск, что это случится, и тем критичнее для бизнеса такое падения.

Проблемы монолита

- **Увеличение нагрузки на модуль приложения создает нагрузку на БД, от чего может замедлиться вся БД, и в целом все приложение.**

Чем больше монолит, тем выше риск, что это случится, тем выше нагрузка на БД, тем критичнее для бизнеса деградация.

- **Язык программирования всего приложения может не подходить для специфичных модулей.**

Например, модуль расчета CPU-bound и реализация на С работала бы быстрее. Чем больше монолит, тем больше таких модулей появляется.

Проблемы монолита

По сути основные проблемы монолита заключаются в излишней связности модулей между собой:

- единый артефакт деплоя
- единое хранилище (БД)
- единый фреймворк
- язык программирования и библиотеки
- единая кодовая база

Вопросы?

 если есть вопросы

 если вопросов нет

Микросервисная архитектура

Что такое микросервис?

Микросервисная архитектура – это способ проектирования сложного приложения как набора небольших и независимых **сервисов**, которые реализуют конкретную функциональность части приложения, разрабатываются и развертываются независимо друг от друга.

Микросервисом называют независимо развертываемый компонент, предоставляющий сервис для реализации конкретной функциональной части приложения (ГОСТ Р70860-2023).

Микросервис не определяется каким-то конкретным размером или формальным требованием. «Правильный» размер микросервиса ровно такой, который не создает проблем.

Микросервисная архитектура

Плюсы:

- Масштабируем только те сервисы, которые этого требуют
- Внесение изменений в один микросервис проще, чем в монолит
- Быстрый билд/деплой одного микросервиса
- Используем подходящие под задачу технологии
- Эволюционное развитие приложения

Микросервисная архитектура

Плюсы:

- Сильные границы для модулей
- Независимый деплой
- Использование подходящих технологий

Минусы:

- Распределенные системы сложнее: удаленные вызовы по сети всегда медленнее и могут упасть
- Поддержание консистентности данных становится сложнее
- Операционная (инфраструктурная) сложность. Требуется отдельная платформа для развертывания, мониторинга, конфигурирования и т.д. большого количества сервисов

<https://martinfowler.com/articles/microservice-trade-offs.html>

Плюсы и минусы микросервисов

Микросервисы предоставляют преимущества...

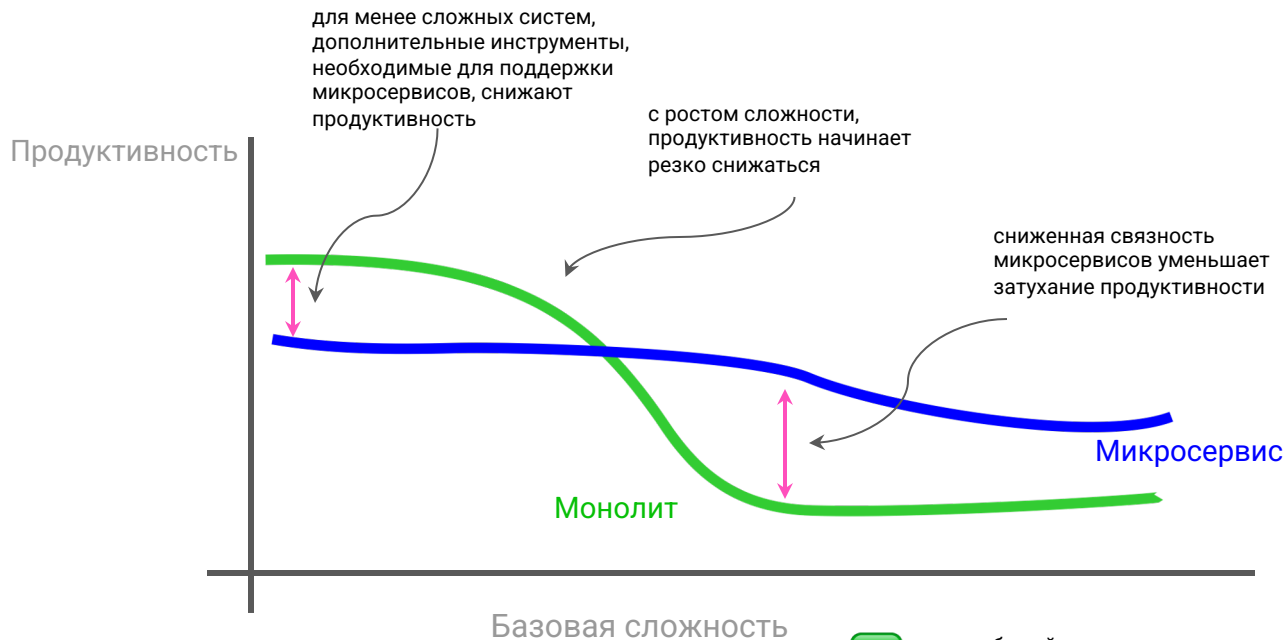
- ✓ Сильные границы модулей: микросервисы усиливают модульную структуру, что особенно важно для больших команд.
- ✓ Независимое развертывание: простые сервисы проще развертывать, и поскольку они автономны, меньше вероятность того, что они вызовут сбой в работе системы, если они выйдут из строя.
- ✓ Разнообразие технологий: с помощью микросервисов вы можете совмещать несколько языков, фреймворков разработки и технологий хранения данных.

... но какой ценой

- ✗ Распределение: распределенные системы сложнее программировать, поскольку удаленные вызовы медленные и всегда подвержены риску сбоя.
- ✗ Согласованность в конечном счете (Eventual Consistency): поддерживать сильную согласованность чрезвычайно сложно для распределенной системы, что означает, что каждый должен управлять согласованностью в конечном счете.
- ✗ Операционная сложность: вам нужна зрелая операционная команда для управления большим количеством сервисов, которые регулярно переразвертываются.

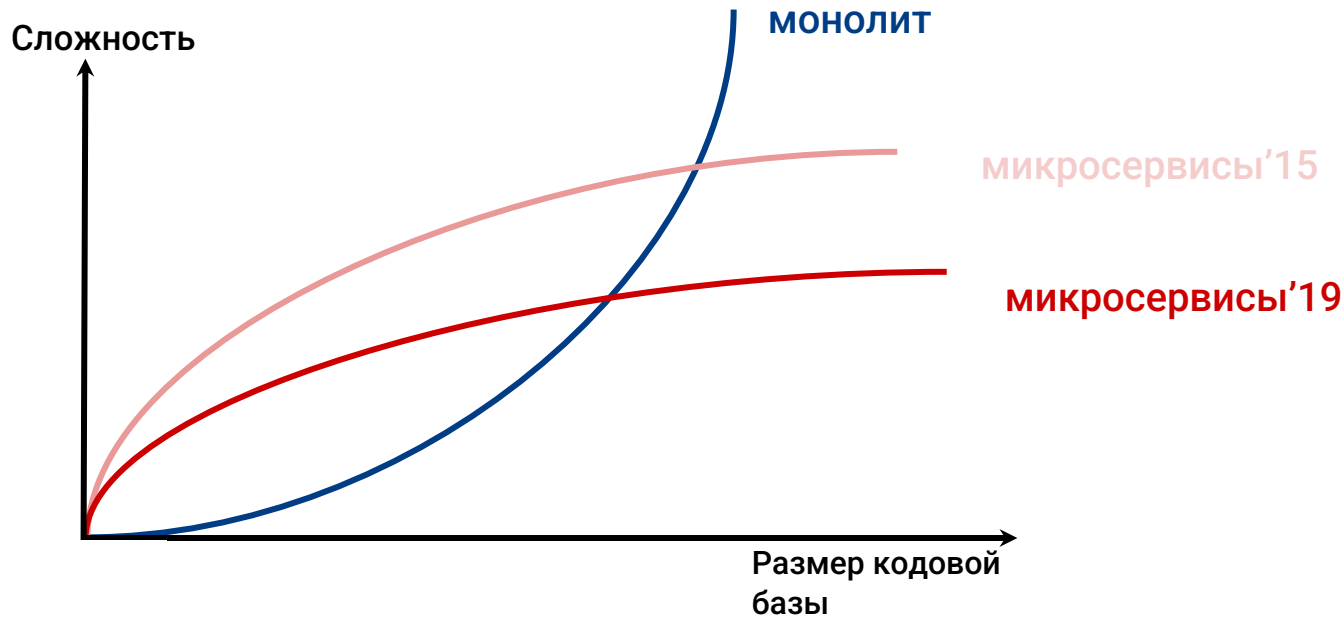
<https://martinfowler.com/articles/microservice-trade-offs.html>

Микросервисы и монолит



но не забывайте, что навыки команды перевешивают любой выбор между монолитом и микросервисом

Микросервисы и монолит





Когда микросервисная
архитектура не подойдет?

Когда микросервисы могут не подойти?

- В проектах, в которых предметная область не до конца ясна.
- Коробочные решения - ПАК, антивирус и т.п.

Микросервисная архитектура

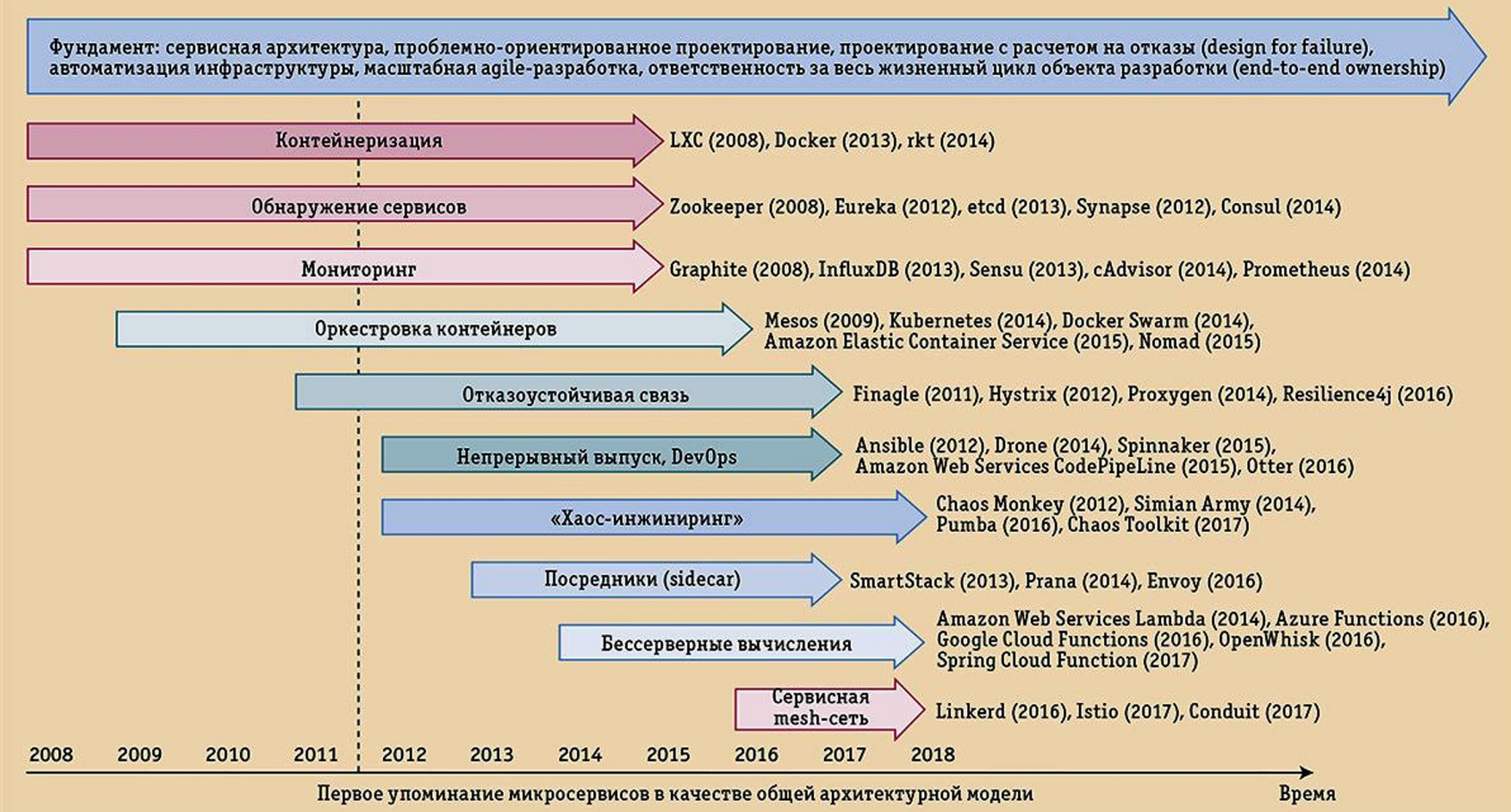
- Микросервисная архитектура не должна и не является самоцелью
- Микросервисная архитектура не подходит всем
- Решение об адаптации подходов микросервисной архитектуры должно быть продуманным, исходя из потребностей, например, если текущий архитектурный подход приводит к проблемам

Микросервисная архитектура

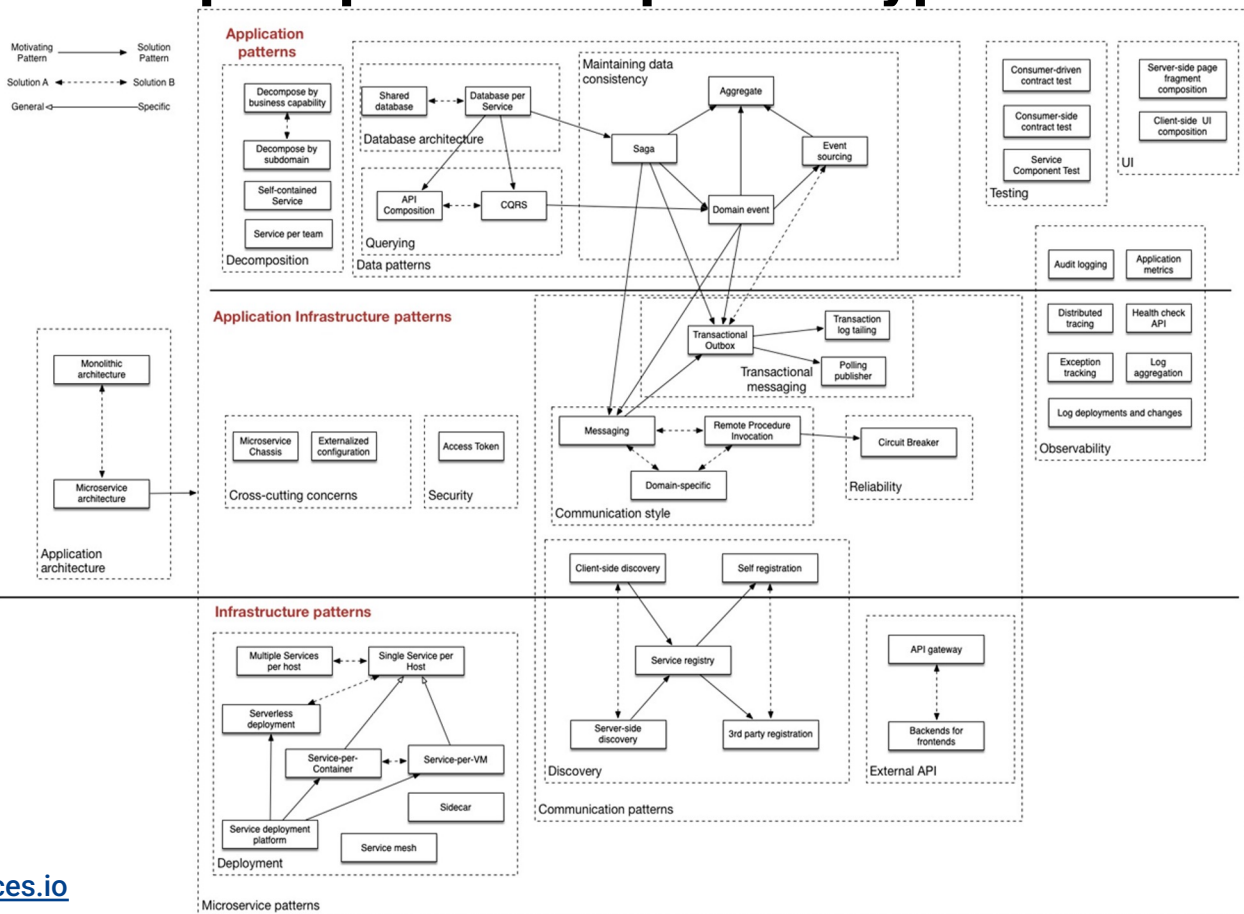
При очевидных плюсах микросервисной архитектуры, таких как выделенные сильные границы модулей, независимый деплой и использование подходящих под задачу технологий, **основной проблемой микросервисов становится сложность:**

- проектирование и эксплуатация распределенных систем сложнее
- поддерживать консистентность данных крайне сложно
- требуется автоматизированное управление развертыванием, мониторингом, конфигурированием и т.д. для большого количества сервисов

Как развивались паттерны микросервисной архитектуры



Паттерны микросервисной архитектуры



Вопросы?

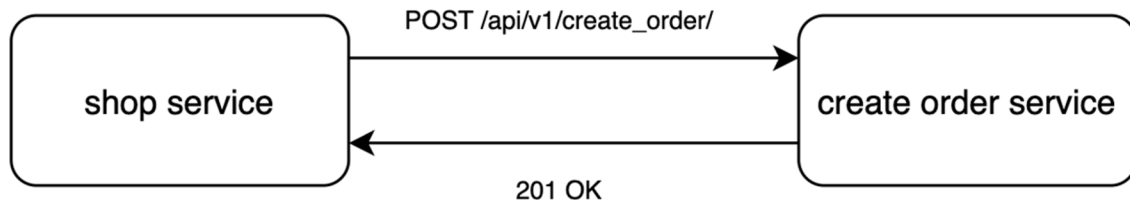
 если есть вопросы

 если вопросов нет

Инфраструктура микросервисов

Кейс

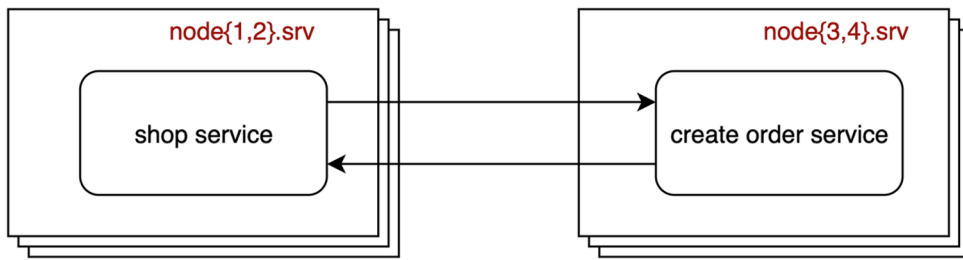
Представим, что у нас есть интернет-магазин в виде монолита на php. Выделим (нано)сервис, который занимается оформлением заказа.



Инфраструктура усложнилась

Сервис «оформление заказа» необходимо размещать на каких-то серверах, необходимо думать о горизонтальном масштабировании и отказоустойчивости (отключать трафик в случае падений и т.д.).

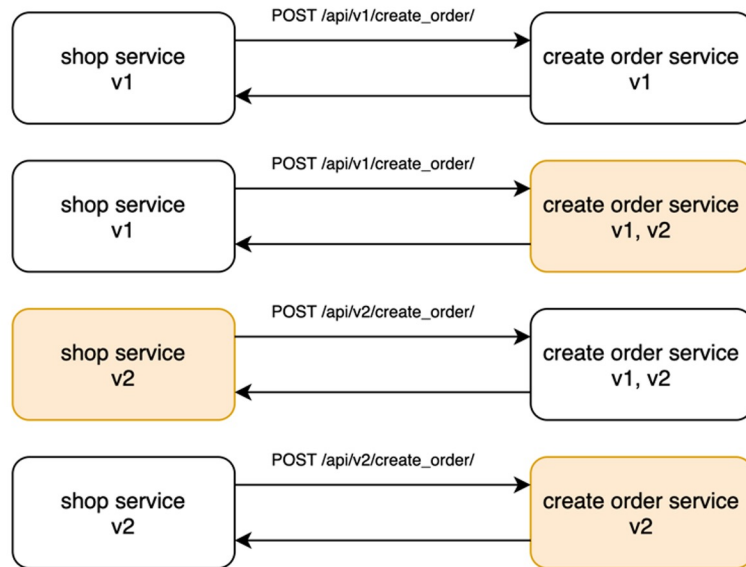
Кроме того, для этого сервиса необходимо написать и организовать pipeline для сборки и деплоя.



Релизить стало сложнее

Поскольку деплоятся сервисы независимо друг от друга, то нельзя просто взять и поменять API со сломом обратной совместимости.

Необходимо поддерживать две версии API в одном коде.



Инфраструктурные проблемы

- Как на одной машине запустить процессы с разным окружением?
- Как сделать обнаружение сервисов?
- Как ограничить ресурсы на сервис?
- Как можно автоматически перекидывать сервисы в случае аварии на машине?

И т.д.

Инфраструктура для микросервисов

Обычно решается с помощью инструментария оркестратора рабочей нагрузки или провайдера облачной платформы.

Примеры:

- **Docker (Podman, CRI-O)** – контейнеризация
- **Kubernetes** – оркестрация
- **Istio** – service mesh



Вопросы?

 если есть вопросы

 если вопросов нет

Тестирование микросервисов

Тестирование микросервисов

- Как тестировать сервисы/микросервисы, если их много?

Раньше(в монолитах) тесты были юнит-тестами, то теперь тесты стали интеграционными. Для проверки пользовательского сценария необходимо локально (или на стендах) поднимать и настраивать еще дополнительный сервис.

- Как тестировать API и контракты?

Тестирование микросервисов

Для тестирования микросервисов используют паттерны:

- E2e тесты
- Consumer driven contracts

Примеры инструментов:

- PACT – для CDC
- Puppeteer, Selenium, Cucumber– для e2e тестов

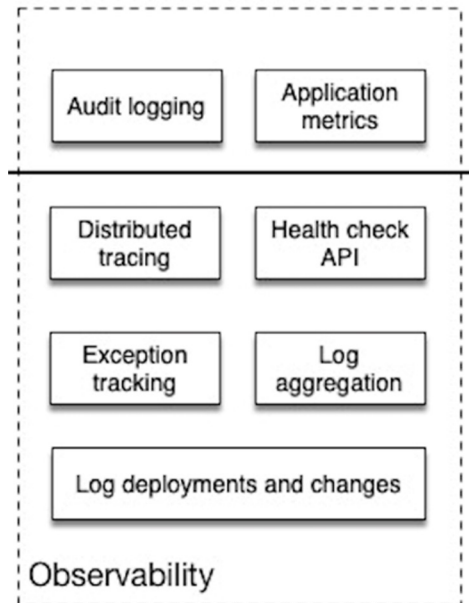
PACT



Инструменты наблюдаемости

Наблюдаемость (Observability)

- Как понять, какие сервисы задействовал клиентский запрос?
- Как найти узкое место, если сервисов много?
- Как понять, сервис жив или нет?
- Как понять, как много и каких ошибок в коде?



Наблюдаемость (Observability)

Инструментирование кода и использование соответствующих инструментов

Примеры:

- Prometheus – для сбора метрик
- Grafana – для визуализации
- ELK – для визуализации и логирования
- Jaeger – распределенные трассировки
- Sentry – перехватывание исключений



30



Внешний API

Внешний API

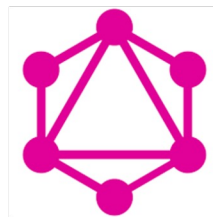
- Как дать доступ клиентам приложения к конкретным микросервисам?
- Как аутентифицировать и авторизовать клиентские запросы?
- Что делать, если для API микросервиса используется не web-friendly протокол?
- Как балансировать нагрузку?

Внешний API

Обычно решается с помощью специальных API Gateway-ев и/или фасадов для клиента.

Примеры инструментов:

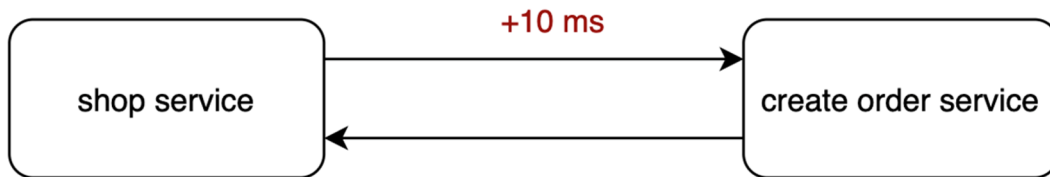
- Nginx
- Traefik – api gateway/k8s ingress controller
- Kong - API Gateway
- Envoy – reverse proxy
- GraphQL



Взаимодействие микросервисов

Время ответа увеличилось

Время ответа увеличилось из-за того, что время локального вызова и время по сети значительно отличаются.

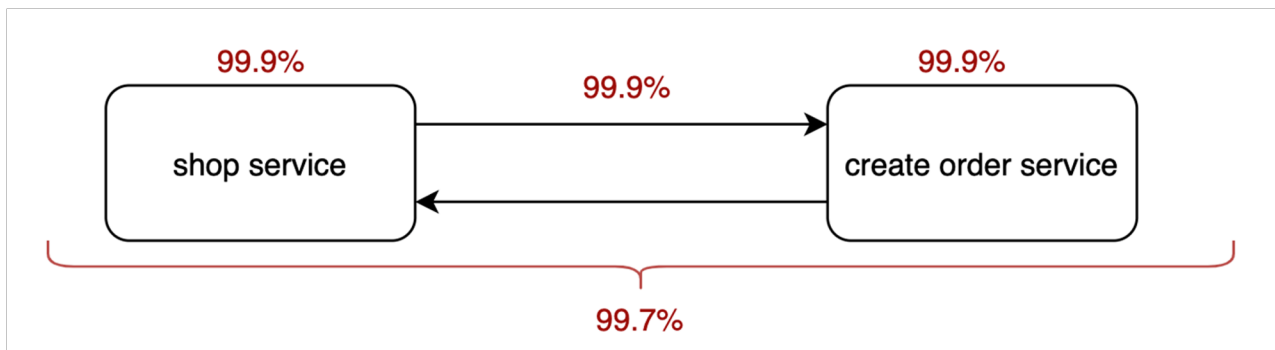


RPC != вызов функции

	Время доступа.	Для масштаба
L1 кэш	0.5 нс	1 с
L2 кэш	7 нс	14 с
Вызов функции	100 нс	3.5 мин
Сжать 1Kб	3000 нс.	1ч 40 мин
Переслать 2Kб по 1GBps сети	20 000 нс	16ч 40 мин
Пересылка пакета внутри ДЦ	500 000 нс	11 дней 14 часов
Пересылка пакета по сети	150 000 000 нс.	13 лет

Доступность уменьшилась

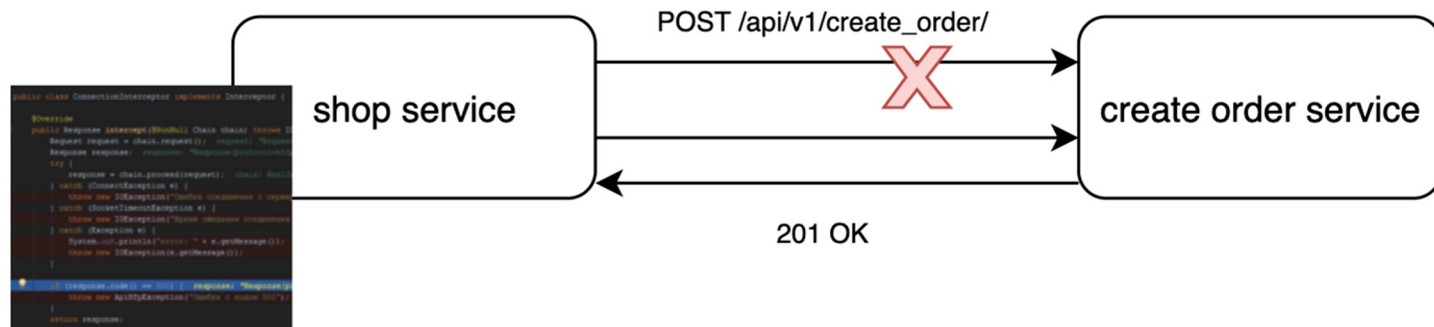
Доступность = доступность shop * доступность сети * доступность create order



Разработка стала сложнее

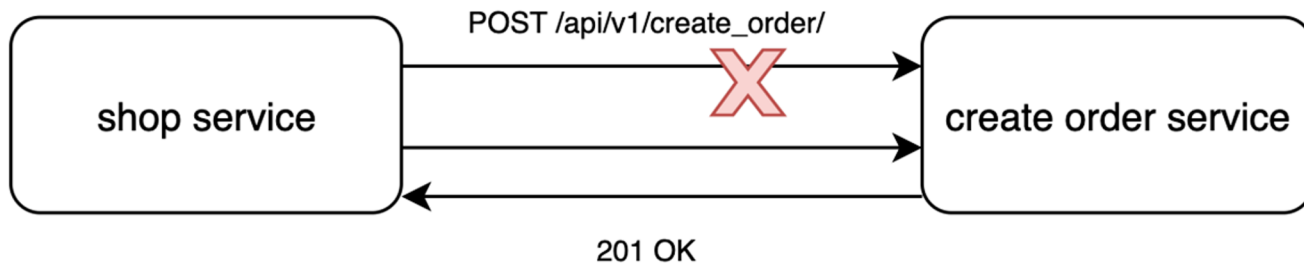
Если раньше IDE подсказывала, какой тип параметров передавать, то теперь спецификация API хранится вне кодовой базы (в вики или swagger).

Кроме того, если раньше просто выполняли код, то теперь т.к. это RPC, приходится писать еще код вокруг вызовов, обрабатывающий сетевые ошибки, думать про таймауты и т.д.



Отказоустойчивость поддерживать сложнее

Необходимо корректно обрабатывать ситуации, когда один запрос может быть отправлен два раза, либо запросы отправлены в разном порядке.



Взаимодействие сервисов

Для увеличения отказоустойчивости для микросервисной архитектуры характерно использование асинхронных протоколов взаимодействия.

Примеры протоколов:

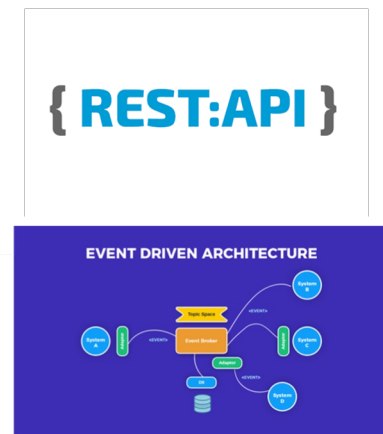
- Синхронные – gRPC, RESTful, OpenAPI
- Асинхронные – Kafka, RabbitMQ

Паттерны:

- Event Driven Design, Domain Events



{ **REST:API** }



Взаимодействие сервисов

Сейчас наблюдается тенденция выноса взаимодействия сервисов на инфраструктурный уровень. Разработчики не должны думать о том, как взаимодействуют сервисы.

Паттерны:

- Service mesh (например, Istio)
- Microservices Chassis (шаблоны сервисов и абстрактные коннекторы)



Вопросы?

 если есть вопросы

 если вопросов нет

Функциональная декомпозиция

Декомпозиция сервисов



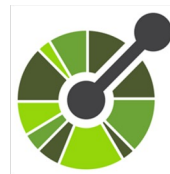
Как разделить приложение на сервисы?

Декомпозиция сервисов

Для декомпозиции приложения на микросервисы используется обычно как основа DDD(Domain-driven design).

Инструменты и паттерны:

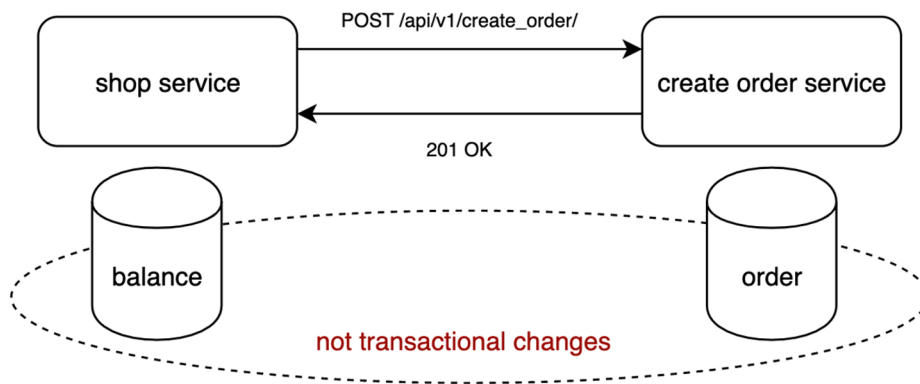
- Bounded Context – стратегический паттерн из DDD
- Event Storming – как метод/практика описания предметной области и бизнес-процессов
- Microservice as aggregate – паттерн декомпозиции
- API design first (OpenAPI, GraphQL Scheme и т.д.)



Данные и микросервисы

Транзакционность поддерживать сложнее

В случае проблем с сетью, мы не сможем обеспечить достаточно просто консистентность данных, находящихся в двух сервисах.



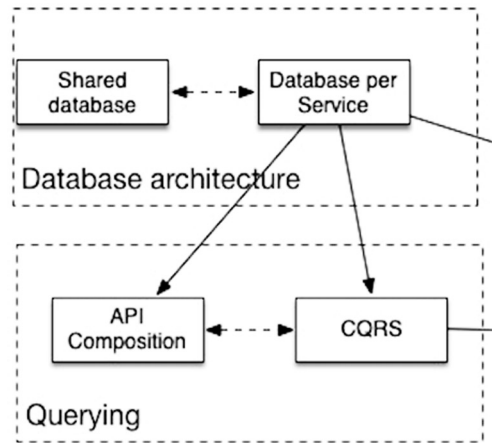
Поддержание консистентности данных

Поскольку строгая консистентность – очень дорого, то обычно используется eventual consistency.

Но даже eventual consistency не так просто поддерживать.

Декомпозиция данных

- Как хранить данные?
- Можно ли другим микросервисам ходить в чужую БД?
- Что делать, если нужно смерджить данные из разных микросервисов?

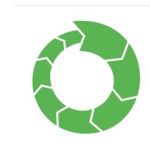


Декомпозиция данных

Чаще всего используется паттерны:

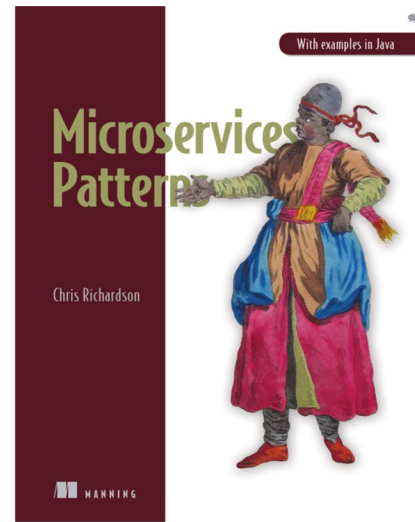
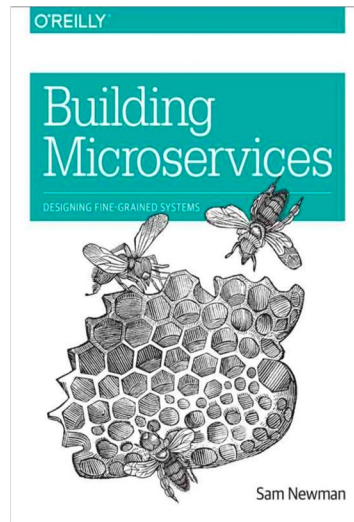
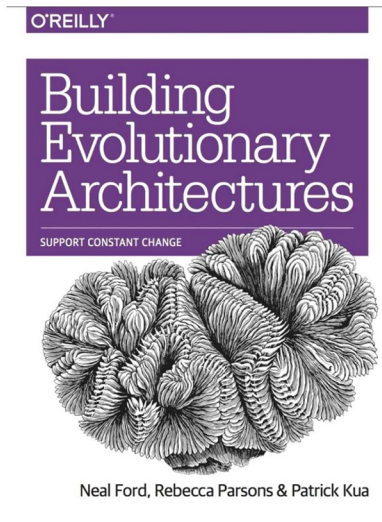
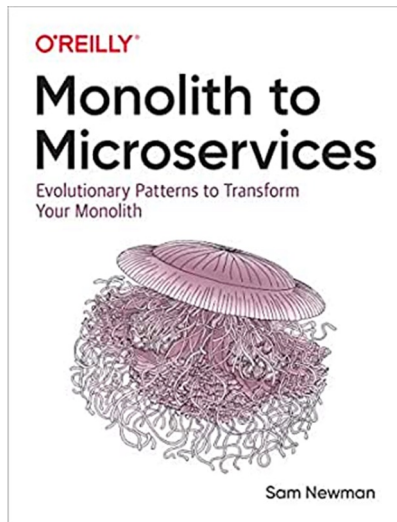
- 1 БД на 1 сервис
- Границы транзакционности проходят по сервису

Иногда используют Event Sourcing + CQRS.



CQRS
Command Query Responsibility Segregation

Книги



Вопросы?

 если есть вопросы

 если вопросов нет

Проектная работа и домашние задания

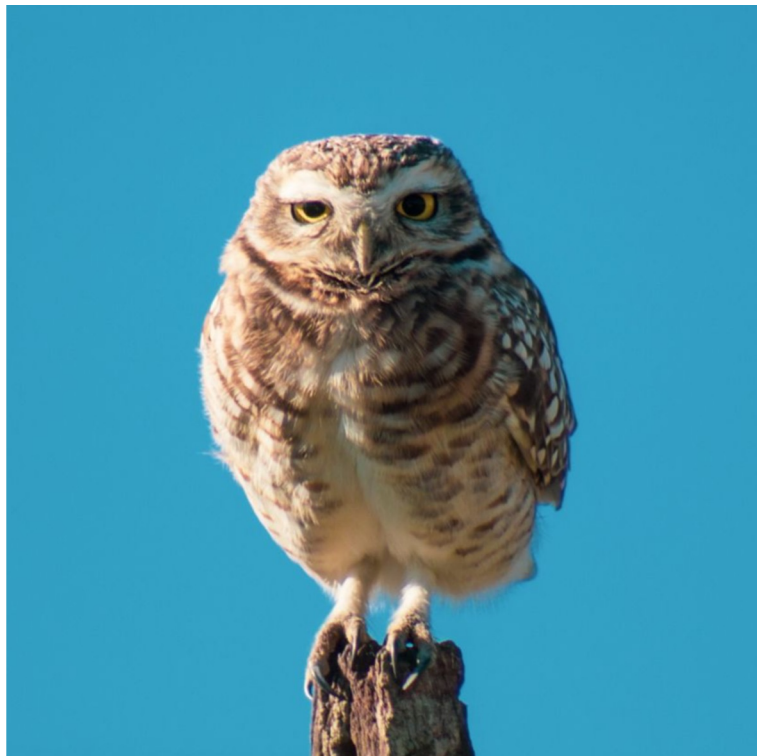
Рекомендации

Делать домашние задания(ДЗ)
и проектную работу!

ДЗ есть в двух вариантах:

- с кодом
- без кода

Все сделанные
ДЗ с кодом -- > проектная работа.



Проектная работа

1. Основная цель вашего проекта — продемонстрировать те знания, которые вы получили.
2. Тема проекта будет в сертификате.
3. На защите проекта вам надо будет подготовить презентацию проекта – рассказать и продемонстрировать основные пользовательские сценарии, описать архитектуру решения, рассказать о примененных паттернах и инструментах.
Созданным решением можно пользоваться.



**Сегодня ДЗ
не будет!**



**Подведём
итоги занятия**

Слайд с тезисами

Подведем итоги

1. Определения архитектуры ПО:

- набор арх. решений
- структура

На практике существуют оба подхода. Неотъемлемой частью архитектуры будет понимание траектории развития и границ проекта.

2. Есть всегда плюсы и минусы у монолитной и микросервисной архитектур.

Переход к микросервисной архитектуре должен быть осознанным, обоснованным. Зачем микросервисы? Какие проблемы хотим решить микросервисной архитектурой?

Цели вебинара

Отправьте в чат номера достигнутых целей

1. Узнали плюсы и минусы монолитной архитектуры.
2. Узнали плюсы и минусы микросервисной архитектуры.
3. Ознакомились с разделами курса, проблематикой и паттернами микросервисной архитектуры.

Рефлексия



Что нового вы узнали из занятия?



Какие проблемы микросервисной архитектуры вы можете назвать?



В каких случаях будете выбирать монолитное решение?

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

Приглашаем на следующий вебинар: “Атрибуты качества и представления архитектуры”



Ссылка на вебинар будет
в ЛК за 15 минут



Материалы
к занятию в ЛК —
можно изучать



Обязательный материал
обозначен красной лентой

Архитектура

